

Frontend-разработчик

Введение в курс

Браузер · HTML · CSS · JavaScript

Первая пара

Цели первой пары

- Понять задачи frontend-разработчика и устройство веб-страницы
- Разобраться с форматом занятий и домашними работами
- Зафиксировать правила Git, Pull Request и проверки
- Подготовить окружение к следующему занятию

Короткая диагностика группы

Поднять руку или ответить в форме

- Кто уже писал программу больше одного файла?
- Кто пользовался терминалом и запускал проект из командной строки?
- Кто создавал ветку, делал commit, push и Pull Request?
- Кто понимает, что происходит после ввода адреса сайта в браузере?

Ответы не влияют на оценку. Они определят темп первых занятий.

Договорённости

Как общаемся, сдаём работу и получаем обратную связь

Коммуникация

- Общие объявления и быстрые вопросы: чат курса
- Персональные вопросы: почта университета
- Вопросы по конкретному коду: комментарии в Pull Request
- Если ответа нет больше семи дней, сообщение можно повторить с упоминанием преподавателя

Коммуникация



Вопрос по коду должен воспроизводиться

В сообщении нужны

- Ссылка на ветку или Pull Request с текущим состоянием
- Что ожидалось и что произошло фактически
- Полный текст ошибки и шаги для воспроизведения
- Что уже проверили самостоятельно

Скриншот без кода и фраза «не работает» не дают возможности помочь.

Как устроено обучение

- На занятии разбираем концепцию и собираем небольшой работающий пример
- После занятия материалы и код появляются в репозитории курса
- Домашняя работа закрепляет один конкретный навык
- Проверка проходит через ревью Pull Request

Нулевая работа: Git до frontend

- Создать публичный репозиторий и заполнить README с ФИО и группой
- Создать отдельную ветку, добавить небольшой файл и сделать несколько осмысленных commit
- Отправить ветку на GitHub и открыть Pull Request в основную ветку
- Исправить замечание в том же Pull Request

Проверка ДЗ

- ДЗ выполняется в своем собственном публичном репозитории на github и только в нем
- Никакие другие репозитории в этом не участвуют
- В README.md репозитория указать группу, ФИО

Проверка ДЗ

- При сдаче ДЗ, можно попросить его сразу принять если работа выполнена удовлетворительно
- Или наоборот не принимать пока исправление замечаний не приведет к существенному повышению качества работы
- После двух и более итераций проверки (возвратов на доработку), ДЗ принимается сразу, в том случае если качество работы является минимально приемлемым
- Проверка исправлений замечаний в принятой работе не предусмотрена и осуществляется исключительно по личной договоренности

Проверка ДЗ

- Делать задания лучше после лекций, к которым они относятся
- Некоторые ДЗ связаны друг с другом
- Их нужно сдавать последовательно и только после того, как принято прошлое связанное ДЗ
- Сдавать параллельно можно только не связанные задания

Правила при разработке и сдаче ДЗ

- ДЗ выполняется в отдельной ветке и сдается в формате ссылки на Pull Request (PR/ПР). ПР создается из ветки ДЗ в основную ветку
- Каждый ПР содержит все файлы задания и только их. Даже если ДЗ связаны
- При возврате задания с доработки новый ПР создавать не нужно. Одно ДЗ = один ПР

Если сдали не ссылку на ПР или в ПР есть лишние или недостающие файлы, отправляю сдавать в нужном формате

ПР можно мерджить (merge) после того, как работа принята

Red Flags при сдаче ДЗ

- Сдаете все ДЗ в последние 2 недели -> комиссия
- Сдаете все ДЗ в последний день и "у меня по остальным предметам 5" -> все ещё комиссия
- Код не собирается / не запускается -> "дома работало" не принимается
- Игнорируются замечания по ревью -> одни и те же ошибки в каждом последующем ПР

Вопросы?

Сейчас лучше уточнить формат, чем переделывать сдачу

Критерии оценки

1. Работоспособность (0–5)

0–5

не собирается / не запускается — 0
основной функционал работает, есть баги — 3
все заявленные сценарии работают, нет критичных багов — 5

2. Качество кода (0–4)

0–4

нарушены договорённости, мержить нельзя — 0
базовая структура есть, но много стилистики/дублей — 2
чистый код, разумная архитектура, соблюдён стиль — 4

3. Бонус / штраф (–1…+1)

–1…+1

Принято: итог >= 6

Просрочка дедлайна: -1 балл каждую неделю

Материалы курса

GitHub-репозиторий курса

https://github.com/wordyarc/dvfu-frontend

- Презентации и примеры с занятий
- Заготовки проектов и упражнения
- Описание домашних работ и критерии проверки

Заменить адрес на реальный перед занятием

Что такое frontend

Интерфейс, с которым человек решает свою задачу

Frontend: что видит и делает пользователь

- Структура страницы: заголовки, формы, навигация
- Внешний вид: расположение, размеры, цвета
- Поведение: ввод, клики, проверка и обратная связь
- Работа с данными: запрос, ожидание, результат, ошибка

От документов к приложениям

1989

Тим Бернерс-Ли предлагает проект Web в CERN.

1990

Появляются первый браузер и веб-сервер.

Сегодня

В браузере работают почта, редакторы, карты и личные кабинеты.

Три основы веб-интерфейса

HTML

Смысл и структура

Что это: заголовок, ссылка, поле ввода?

CSS

Представление

Как это расположено и выглядит на разных экранах?

JavaScript

Поведение

Что происходит после ввода, клика или ответа сервера?

Браузер — среда выполнения

- Получает ресурсы и отображает страницу
- Исполняет JavaScript и обрабатывает события
- Даёт API: сеть, хранилище, мультимедиа и другие возможности
- Ограничивает доступ к устройству и данным

Открыли браузер и ввели адрес. Что дальше?

https://example.com/catalog

Браузер пока знает только адрес. Страницы у него ещё нет.

Нажали Enter. Как на экране появится страница?

Попробуем восстановить путь по шагам

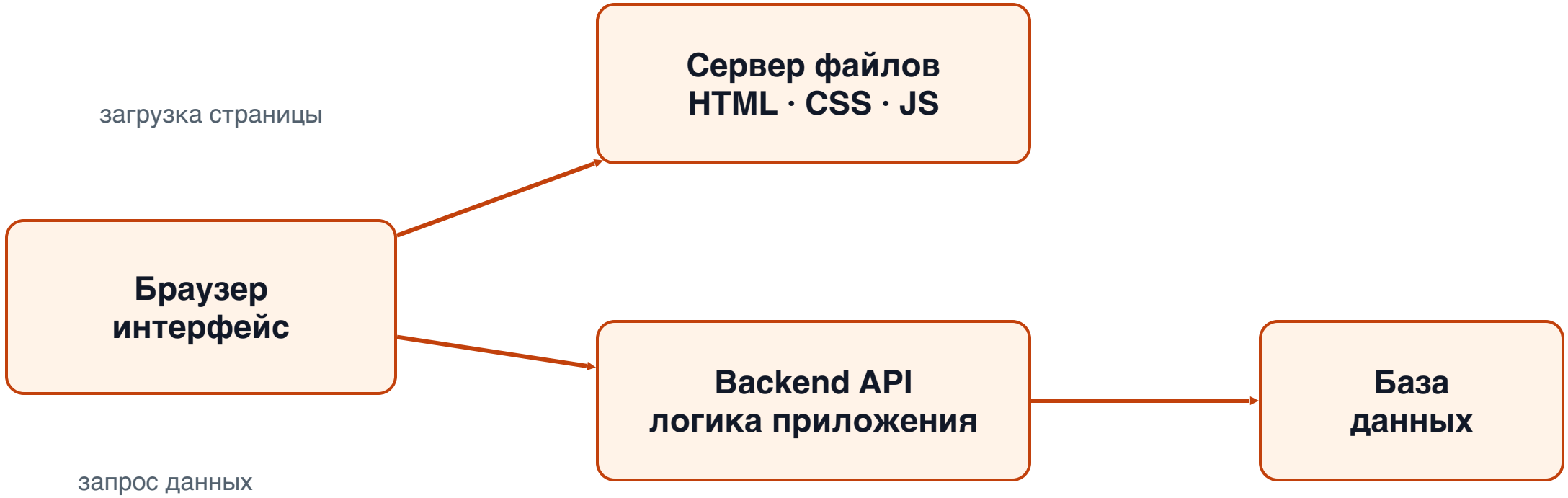
От адреса до ответа сервера

- DNS помогает найти IP-адрес сервера по имени сайта
- Браузер устанавливает соединение; HTTPS защищает обмен
- Отправляет HTTP-запрос к ресурсу /catalog
- Сервер возвращает HTTP-ответ: статус, заголовки и тело

Ответ получен. Как появляются пиксели?

- Браузер разбирает HTML и строит дерево документа DOM
- По ссылкам загружает CSS, JavaScript и изображения
- Применяет стили, рассчитывает размеры и положение элементов
- Рисует страницу; JavaScript может менять её и реагировать на события

Кто с кем общается



Ответы возвращаются обратно по тем же связям

Как страница становится интерфейсом

Документ

Браузер разбирает HTML и строит DOM — дерево элементов.

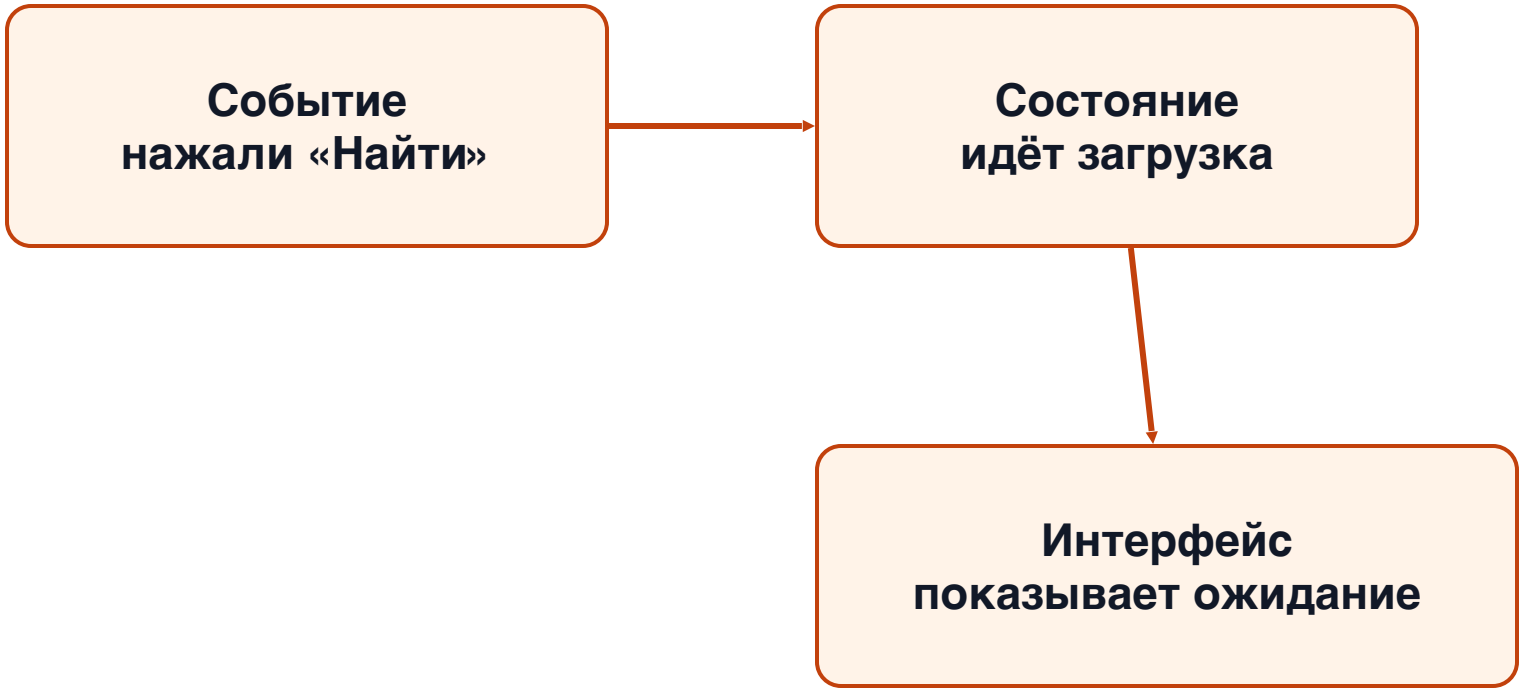
Отображение

Стили определяют вид. Браузер рассчитывает размеры и рисует страницу.

Изменения

JavaScript реагирует на события и может менять DOM и стили.

Клик — это только начало



Пришёл ответ → меняем состояние → обновляем экран

У поиска больше одного состояния

До результата

Ещё не искали

Загружаем: действие принято, нужно подождать.

После ответа

Есть результаты

Ничего не найдено: предложить изменить запрос.

При проблеме

Ошибка запроса

Объяснить ситуацию и дать повторить действие.

Frontend и backend: кто за что отвечает

Frontend

Помогает заполнить форму.

Показывает данные, ожидание и ошибки.

Проверяет ввод для удобства.

Backend

Проверяет права и данные повторно.

Применяет правила бизнеса.

Сохраняет изменения.

Одна страница — разные экраны

- Ширина окна меняется: телефон, планшет, ноутбук
- Текст может стать длиннее, шрифт — крупнее
- Мышь, касание и клавиатура дают разные сценарии
- Адаптивность сохраняет удобство и порядок содержания

Доступность — часть работающего интерфейса

- Управление с клавиатуры и видимый фокус
- Подписи полей и понятные сообщения об ошибках
- Достаточный контраст; смысл не только через цвет
- Семантические элементы и описания важных изображений

Пользователь не обязан ждать

- Большие изображения и скрипты загружаются дольше
- Тяжёлая работа в браузере мешает интерфейсу отвечать
- Медленная сеть — обычный сценарий, его надо проверять
- Сначала измеряем проблему, затем оптимизируем

DevTools: посмотреть внутрь страницы

Elements

Посмотреть структуру и стили.

Временно изменить вид элемента.

Console

Увидеть сообщения и ошибки JavaScript.

Network

Посмотреть запросы, ответы и время загрузки.

Рабочие инструменты

Редактор + Git

Пишем и организуем файлы.

Храним историю и обсуждаем изменения.

Браузер

Запускаем интерфейс.

Проверяем поведение и ищем ошибки.

Node.js + npm

Запускаем инструменты разработки.

Устанавливаем зависимости и собираем проект.

А где TypeScript и фреймворки?

- TypeScript добавляет проверку типов к JavaScript
- Компоненты помогают делить интерфейс на части
- Библиотеки и фреймворки помогают строить приложения
- HTML, CSS, JavaScript и браузер остаются основой

Мини-задача: поиск по каталогу

В парах · 3 минуты · без кода

Пользователь ввёл «ко», затем быстро дописал «кофе».

- Какие состояния интерфейса нужно показать?
- Что увидит пользователь, если совпадений нет?
- Что делать, если сеть пропала?

Усложнение: ответы пришли не по порядку

1. Отправили запрос «ко»
2. Отправили запрос «кофе»
3. Получили ответ на «кофе»
4. Получили ответ на «ко»

Какой результат должен остаться на экране?

Разбор: актуален последний запрос

- Показываем результаты для текущего запроса «кофе»
- Устаревший ответ на «ко» не должен затереть результат
- Можно отменять старый запрос и проверять актуальность ответа
- Задержка перед поиском снижает число запросов, но не решает всё

Что будем уметь к концу курса

- Собрать семантическую и адаптивную страницу
- Реализовать формы и интерактивное поведение
- Получить данные из API и обработать состояния
- Найти проблему через DevTools и проверить исправление
- Организовать проект и представить работу через Git

К следующей паре

- Подготовить редактор кода, браузер и Git
- Проверить доступ к репозиторию курса
- Выбрать знакомый сайт и открыть его DevTools
- Найти форму, состояния загрузки и ошибки; записать наблюдения

Куда смотреть между занятиями

- MDN Web Docs — основы Web и справочник
- Документация браузера — DevTools
- Документация технологии — когда добавляем её в проект
- Репозиторий курса — задания, материалы и договорённости

План курса

01 Web, инструменты и Git

02 HTML: структура, семантика, формы

03 CSS: стили, раскладка, адаптивность

04 JavaScript: язык, DOM, события

05 API, асинхронность и состояния интерфейса

06 TypeScript и компонентный подход

07 Качество, сборка и итоговый проект